

Adevr.ai Architecture Document

Version: 1.0
Date: 2026-03-01
Status: In Review

1. Executive Summary

Adevr.ai is a public-facing data journalism platform focused on providing reliable, up-to-date socio-economic indicators and news for Moldova. Architected as a **clean, modular monolith**, it leverages a modern web stack consisting of **Next.js 14** for the frontend and API, **Supabase** (PostgreSQL) for the data backend, and **Vercel** for deployment and hosting.

The system is designed around a core principle: a strict separation between raw data ingestion and curated data presentation. External data, including economic indicators and news articles, is processed through idempotent, validated ingestion pipelines. All user-facing interfaces—from interactive indicator charts to the live news feed—read exclusively from the trusted, sanctioned database layer in Supabase. This ensures data integrity, traceability, and consistency.

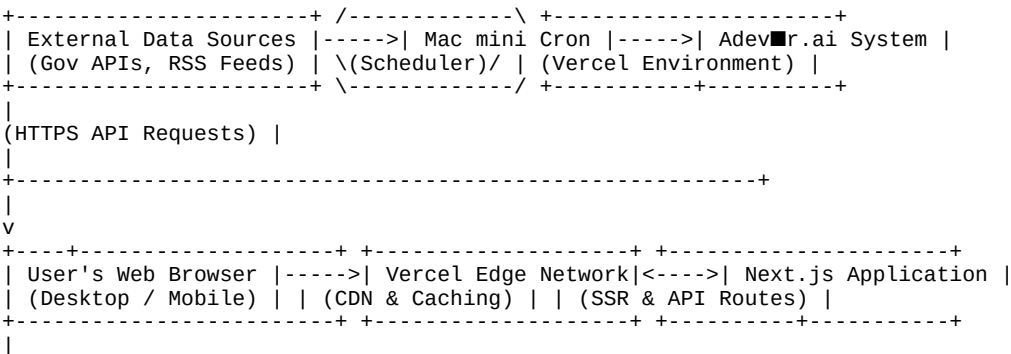
Key implemented features include a dynamic chart platform, a dashboard with real-time statistics, a comprehensive news aggregation and analysis pipeline, and a responsive mobile-first user interface. The operational model relies on automated scripts for data ingestion, triggered by an external scheduler, and a safe, preflight-checked deployment process to Vercel.

2. Principles

- Modular Monolith First:** Build with clear boundaries between components (UI, API, Services, Data) within a single application codebase. This provides development velocity while allowing for future, targeted extraction of services only when necessary.
- Data Integrity is Paramount:** User-facing applications **must not** consume data from external sources directly. All data must pass through a validation and ingestion process into the Supabase database. This is the "single source of truth."
- Build for Correctness & Traceability:** Prioritize data quality, governance, and auditability. Implement features like data change logs and metadata tracking before focusing on complex AI or scaling initiatives.
- Idempotent & Resilient Operations:** All data ingestion and processing scripts must be safely re-runnable without causing data duplication or corruption. Service layers should employ resilient data retrieval strategies (e.g., fallbacks) to handle inconsistencies.
- Secure by Default:** Implement security at each layer, from token-protected API endpoints to database row-level security and strict data validation.

3. System Context

Adevr.ai operates within a simple, robust ecosystem designed for reliability and scalability.



```

(DB Connection over SSL) |
V
+-----+-----+
| Supabase (PostgreSQL) |
| (Data Backend) |
+-----+-----+

```

****Components:****

- ****External Data Sources:**** Third-party government APIs and news RSS feeds that provide the raw data.
- ****Mac mini Scheduler:**** An external cron job runner responsible for periodically triggering the data ingestion scripts (e.g., `news-pipeline.mjs`).
- ****Adevr.ai System (Vercel):**** The core application hosted on Vercel.
- ****Vercel Edge Network:**** Provides CDN caching for static assets and server-side rendering caching for pages.
- ****Next.js Application:**** The monolithic application serving both the user-facing React UI (via App Router) and the backend API routes.
- ****Supabase (PostgreSQL):**** The managed PostgreSQL database that serves as the single source of truth for all user-facing data.

4. Logical Architecture

The application is structured as a modular monolith, with a clear separation of concerns between different logical layers.

```

+-----+
| User Interface (UI) |
| (Next.js App Router: Pages, Layouts, Client Components) |
| [/ , /chart/:id, /news] [Header.js, BottomNav.js, ChartCanvas.js] |
+-----+
| (Client-side Fetch)
+-----+
| API Surface (app/api) |
| (Next.js Route Handlers: Caching, Revalidation, Auth) |
| [/api/dashboard/stats, /api/news/feed, /api/indicators/:slug/series]|
+-----+
| (Server-side Call)
+-----+
| Services Layer (services/) |
| (Business Logic & Data Retrieval) |
| [dashboard-stats.js, indicators.js] |
+-----+
| (Server-side Call)
+-----+
| Core Libraries (lib/) |
| (DB Clients, Auth Helpers, i18n, Validation, Engine) |
| [supabase-server.js, ingest-auth.js, ingestion-validation.js] |
+-----+
|
+-----+
| Data Access Layer (DAL) |
| (Supabase Client executing SQL/RPC calls) |
+-----+

```

- ****UI Layer (`app`, `components`):**** Built with Next.js App Router. It uses a mix of Server Components (for data fetching, e.g., `app/chart/[id]/page.js`) and Client Components (for interactivity, e.g., `components/ChartCanvas.js`).
- ****API Surface (`app/api`):**** A set of RESTful endpoints built with Next.js Route Handlers. These endpoints are the exclusive gateway for the UI to fetch data. They handle caching, data shaping, and forwarding requests to the service layer.
- ****Services Layer (`services`):**** Encapsulates business logic for data retrieval and manipulation. For example, `services/indicators.js` contains logic for fetching indicator data with resilient fallbacks. This layer is called only from the API layer or other server-side processes.
- ****Core Libraries (`lib`):**** Contains shared, reusable modules for concerns like creating Supabase clients, handling authentication, internationalization (i18n), and the core chart rendering engine logic (`lib/engine.js`).
- ****Data Access Layer:**** Abstracted via the Supabase client. All database interactions (queries, upserts, function calls) are performed through this client, primarily within the Services Layer.

5. Data Architecture

The data architecture is centered around a PostgreSQL database managed by Supabase. Schema is managed via versioned SQL migration files.

Key Tables

Indicators & Values:

- ``indicators``: Stores metadata about each economic indicator.
- ``indicator_values``: Stores the time-series data points for each indicator.

```
-- From sql/migrations/20260228_indicator_metadata_and_audit.sql
CREATE TABLE indicators (
  id uuid PRIMARY KEY,
  slug text UNIQUE NOT NULL,
  name text NOT NULL,
  -- ... metadata fields
  source_name text,
  source_url text,
  methodology text,
  update_frequency text,
  unit text,
  is_official boolean
);
CREATE TABLE indicator_values (
  id bigserial PRIMARY KEY,
  indicator_id uuid REFERENCES indicators(id),
  year int NOT NULL,
  value numeric(18, 4),
  -- ... other fields
  UNIQUE (indicator_id, year)
);
```

News Pipeline:

- ``news_sources``: Configuration for each news source to be scraped.
- ``news_items``: Stores individual news articles, with hashes for deduplication.
- ``news_runs``: Logs for each execution of the news pipeline script.

```
-- From sql/migrations/20260301_news_pipeline.sql
CREATE TABLE news_sources (
  id uuid PRIMARY KEY,
  slug text UNIQUE NOT NULL,
  name text NOT NULL,
  rss_url text,
  enabled boolean NOT NULL
);
CREATE TABLE news_items (
  id uuid PRIMARY KEY,
  source_slug text NOT NULL,
  title text NOT NULL,
  url text NOT NULL,
  canonical_url text NOT NULL,
  url_hash text NOT NULL UNIQUE,
  title_hash text NOT NULL,
  published_at timestamptz,
  impact_score numeric(6,2) NOT NULL DEFAULT 0,
  duplicate_group text
);
```

Governance:

- ``data_change_log``: An audit table that logs all modifications to key data tables.

```
-- From sql/migrations/20260228_indicator_metadata_and_audit.sql
CREATE TABLE data_change_log (
  id bigserial PRIMARY KEY,
  table_name text NOT NULL,
  record_key text,
  action text NOT NULL, -- e.g., 'INSERT', 'UPDATE'
  changed_by text,
  reason text,
  payload jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Data Flow

1. ****Ingestion:**** External scripts (``ingest-indicator.mjs``, ``news-pipeline.mjs``) are run, which fetch data from external sources.

2. ****Validation:**** The scripts validate and transform the raw data.
3. ****Upsert:**** Data is written to Supabase tables using `upsert` semantics to prevent duplicates (e.g., on `(indicator\_id, year)` or `url\_hash`).
4. ****Audit:**** Triggers or application logic populates the `data\_change\_log` table upon modification.
5. ****Consumption:**** The Next.js application's API routes query the Supabase tables via the service layer to serve data to the frontend.

6. News Pipeline Architecture

The news pipeline is a discrete, automated system for aggregating, deduplicating, and scoring news articles from various sources.

1. ****Configuration:**** A static JSON file (`config/news-sources.json`) defines the list of RSS feeds to process, including their priority and language.
2. ****Execution:**** The `scripts/news-pipeline.mjs` Node.js script is triggered by the external scheduler.
3. ****Fetching:**** The script iterates through enabled sources, fetches the RSS feeds, and parses the XML into a standardized item format.
4. ****Processing & Hashing:**** For each article, it:
 - Generates a `url\_hash` from the canonical URL for hard deduplication.
 - Generates a `title\_hash` and a `duplicate\_group` hash from the title for soft deduplication analysis.
 - Calculates an `impact\_score` based on keywords (`guvern`, `inflatie`, etc.) and source priority.
5. ****Storage:**** The processed article is inserted into the `news\_items` table using an `upsert` on the unique `url\_hash`. This makes the process idempotent.
6. ****Logging:**** The script logs its execution results (counts of fetched, inserted, deduped items) into the `news\_runs` table.

7. API Surface

The API surface is composed of Next.js Route Handlers. All data is returned as JSON.

`GET /api/dashboard/stats`

- ****Description:**** Returns high-level statistics for the main dashboard.
- ****Caching:**** No store (`Cache-Control: no-store`), data is always fresh.
- ****Response Example:****

```
{
  "totalIndicators": 10,
  "latestNewsItems": 5,
  "systemStatus": "ok"
}
```

`GET /api/indicators/:slug/series`

- ****Description:**** Retrieves the full time-series data and metadata for a specific indicator.
- ****Caching:**** Statically generated at build time, with incremental static regeneration if configured.
- ****Response Example**** (`/api/indicators/inflation/series`):

```
{
  "indicator": {
    "slug": "inflation",
    "name": "Inflaie",
    "source_name": "Biroul Naional de Statistic (BNS)",
    "unit": "%"
  },
  "series": [
    { "year": 2020, "value": 0.39 },
    { "year": 2021, "value": 5.11 },
    { "year": 2022, "value": 30.24 }
  ]
}
```

`GET /api/news/feed`

- ****Description:**** Returns a paginated feed of recent news items, sorted by impact score and publication date.
- ****Query Params:**** `?page=1&limit;=20`

- **Response Example:**

```
{
  "items": [
    {
      "id": "...",
      "source_slug": "protv-md",
      "title": "Guvernul a aprobat un nou pachet de legi",
      "summary": "...",
      "url": "...",
      "published_at": "2026-03-01T10:00:00Z",
      "impact_score": 74.00
    }
  ],
  "meta": {
    "currentPage": 1,
    "hasNextPage": true
  }
}
```

`POST /api/ingest/indicator-value`

- **Description:** A protected endpoint for ingesting new indicator data points.
- **Authentication:** Requires a valid `x-ingest-token` in the header.
- **Request Body Example:**

```
{
  "indicator_slug": "gdp",
  "values": [
    { "year": 2025, "value": 280000 }
  ]
}
```

8. UI Architecture

The UI is built using Next.js 14's App Router, which enables a powerful combination of server-side and client-side rendering.

- **Layouts** (`app/layout.js`, `app/ClientLayout.js`): A root server layout handles the overall page structure (``, ``), while a nested client-side layout (`ClientLayout.js`) manages shared state providers (`ThemeContext`, `LangContext`) that require browser interactivity.
- **Server Components:** Most pages (`app/page.js`, `app/chart/[id]/page.js`) are React Server Components (RSCs). They run on the server, can perform direct data access (e.g., reading chart data from files during the build), and generate static HTML for fast initial loads.
- **Client Components** (`use client`): Components requiring state, effects, or browser APIs are marked as Client Components.
- **ChartPageClient.js**: Wraps the chart page, receiving server-fetched data as props but managing client-side logic.
- **ChartCanvas.js**: A key client component that uses a `ref` to an HTML `` element and leverages the browser-based `lib/engine.js` to render the interactive chart visualizations.
- **Styling:** A global CSS file (`app/globals.css`) provides the base styles, following a utility-first or BEM-like approach for component-specific styles.

9. Mobile Architecture

The application is designed mobile-first.

- **Responsive Design:** The UI uses responsive CSS techniques (media queries, flexible layouts) to adapt to various screen sizes.
- **Mobile Navigation:** For small screens, the primary navigation is handled by the `components/BottomNav.js` component, providing an app-like experience with easy-to-tap icons for accessing the Dashboard, Charts list, and News feed.
- **Performance:** Server-side rendering and static generation ensure that mobile devices receive optimized, fast-loading HTML, which is critical for potentially slower network connections.

10. Security & Governance

- **Ingestion Endpoint Protection:** The `/api/ingest/*` routes are protected. The `lib/ingest-auth.js` middleware validates a bearer token (`x-ingest-token`) against a server-side secret (`INGEST_API_TOKEN`), returning a `401` or `403` error on failure.
- **Data Validation:** Ingestion logic includes validation steps to ensure data integrity before it's written to the database.
- **Supabase Security:**
- **Row Level Security (RLS):** Policies are in place to control access to data. For example, public users (`anon` role) have read-only access to specific tables, while service roles (`service_role`) are used for write operations during ingestion.
- **Connection Pooling:** Supabase manages secure, pooled PostgreSQL connections.
- **Audit Trail:** The `data_change_log` table provides a basic but effective audit trail, logging all data modifications for traceability and debugging.
- **No Raw SQL Injection:** The use of the Supabase client library mitigates the risk of SQL injection vulnerabilities by parametrizing queries.

11. Reliability/Observability

- **Health Checks:** A dedicated `/api/health` endpoint provides a simple check on the application's status.
- **Smoke Tests:** A suite of scripts in `scripts/smoke-*.mjs` are used to actively probe live endpoints (`/api/dashboard/stats`, `/api/indicators/inflation/series`, etc.) to verify they are returning valid data. The `smoke-prod-all.sh` script runs the entire suite.
- **Operational Logging:**
- **Ingestion Runs:** The `news_runs` table provides a detailed log of every news pipeline execution, capturing status, counts, and errors.
- **Ingestion Errors:** The `news_errors` table specifically logs errors encountered during the fetching stage of the news pipeline, helping to identify unreliable sources.
- **Vercel Analytics:** Vercel's built-in analytics provide insights into traffic, performance (e.g., Web Vitals), and function errors.

12. Deployment/Operations

- **Platform:** Vercel is the primary deployment and hosting platform.
- **Deployment Script (`scripts/boss-deploy.sh`):** This is the canonical script for production deployments.
 1. It first runs `npm run preflight`, which executes a series of checks (`validate-env.sh`, `check-migrations.sh`).
 2. If the preflight checks pass, it proceeds to run `vercel deploy --prod --yes` to push the new build to production.
- **CI/CD:** The `.github/workflows/ci.yml` file defines a basic continuous integration pipeline that likely runs on pull requests, executing linting and tests.
- **Scheduler:** A Mac mini running `cron` is used as an external scheduler. It is configured to periodically execute the `scripts/news-pipeline.mjs` and `scripts/ingest-indicator-api.mjs` scripts via SSH or a similar mechanism, ensuring data is kept up-to-date.

13. Scaling Plan

The current architecture is well-positioned to scale gracefully.

- **Frontend/API Scaling:** Vercel's serverless infrastructure automatically scales the Next.js application (both page rendering and API functions) in response to traffic. The Edge Network provides a global CDN to handle high request volumes for cached content.
- **Database Scaling:** Supabase offers managed database scaling. The first step under load would be to upgrade the underlying compute instance. For read-heavy workloads, a read replica can be introduced to offload queries from the primary database.

- **Ingestion Scaling:** The ingestion scripts are run on a fixed schedule. If data sources grow significantly, the scheduler can be parallelized, or the scripts can be moved to a serverless function (e.g., Vercel Cron Job, AWS Lambda) that can scale on demand. The idempotent nature of the scripts makes them safe for distributed execution.

14. Risks/Mitigations

- **Risk: External Data Source Unreliability:** An external API may go down, change format, or provide corrupt data.
- **Mitigation:** The ingestion scripts have error handling. The strict separation of ingestion and presentation means a failed ingestion job does not take down the user-facing site; it simply runs on stale data until the next successful run. `news_errors` table tracks faulty sources.
- **Risk: Ingestion Pipeline Failure:** The scheduler could fail or the ingestion script could have a critical bug.
- **Mitigation:** Operational logging (`news_runs`) provides visibility into pipeline health. Smoke tests will fail if data freshness is compromised, triggering alerts.
- **Risk: Ingestion API Security Breach:** The `INGEST_API_TOKEN` could be leaked.
- **Mitigation:** The token is stored as a secret in the environment. It can be rotated easily. Further hardening could involve IP whitelisting on the ingestion endpoint.
- **Risk: Database Performance Bottleneck:** A complex query or high traffic could slow down the database.
- **Mitigation:** The service layer uses targeted, indexed queries. Supabase provides performance monitoring tools. A read replica is a planned scaling path.

15. 30-60-90 Day Roadmap

This roadmap is synthesized from the `ROADMAP.md` and `PROJECT_CONTEXT.md` documents.

30 Days (Immediate Priorities)

- **Objective:** Solidify the Foundation.
- **Key Results:**
- Replace all remaining hardcoded dashboard statistics with values derived from the `dashboard-stats` API endpoint.
- Tighten Supabase Row Level Security policies, particularly removing any fallback `anon` access from server-side data modification paths.
- Add CI gates to the deployment workflow that fail the build if critical indicator data (e.g., inflation) is empty or invalid.

60 Days (Governance Hardening)

- **Objective:** Enhance Data Governance and Reliability.
- **Key Results:**
- Finalize the indicator registry model in the `indicators` table, ensuring all fields (`source`, `frequency`, `methodology`) are populated for all active indicators.
- Implement a versioning workflow or approval process for changes to indicator metadata.
- Expand the `data_change_log` into a user-facing audit trail or internal report, providing a clear history of data provenance.

90 Days (Controlled Innovation)

- **Objective:** Begin development of the controlled AI query layer.
- **Key Results:**
- Design and document a query "intent schema" that can translate natural language questions into structured data queries.
- Implement a prototype of the `/api/ai-query` endpoint with a hardcoded allowlist of tables and columns to prevent unsafe queries.

- Develop a standardized response format for AI queries that includes source citations and data freshness timestamps to maintain trust and transparency.